

FLUTTER

Material 2 — Listas, ListView, Cards e Conteúdo Dinâmico

1. ListView: primeira lista rolável
2. List<String> + ListView.builder
3. Card + ListTile
4. List<Produto>: objetos na interface
5. Cadastro dinâmico: adicionar, remover e Expanded

OBJETIVO DO BLOCO

Fazer a transição entre os exercícios de estado e entrada de dados e a etapa seguinte de Cards interativos e navegação. Aqui, o foco passa a ser representar coleções de dados na interface e permitir que a coleção mude durante a execução.

Progressão central: dados → lista → builder → Card → objeto → lista dinâmica

1. Apresentação do material

Até este ponto, os programas trabalharam principalmente com campos de entrada, controllers, botões, decisões e atualização da interface com `setState()`. Agora o desafio muda: em vez de apresentar apenas um resultado, precisamos exibir vários elementos organizados em uma coleção.

A proposta deste bloco é acrescentar uma ideia nova por vez. Primeiro, uma lista visual simples. Depois, uma lista de dados em Dart. Em seguida, a construção automática dos itens, a apresentação em Cards e, finalmente, uma lista de objetos que pode crescer e diminuir durante a execução.

IDEIA CENTRAL

Uma coleção Dart guarda dados. Um `ListView` apresenta widgets. O trabalho do aplicativo é transformar cada elemento da coleção no widget correspondente.

1.1 Mapa da progressão

Exercício	Conceito novo	Resultado
1	<code>ListView</code>	Lista vertical com rolagem
2	<code>List<String></code> + builder	Itens gerados a partir dos dados
3	<code>Card</code> + <code>ListTile</code>	Lista com apresentação visual organizada
4	<code>List<Produto></code>	Cada item passa a ser um objeto
5	<code>add()</code> , <code>removeAt()</code> , <code>isEmpty</code> , <code>Expanded</code>	Cadastro dinâmico em memória

1.2 Conceitos retomados

- `StatelessWidget` e `StatefulWidget`.
- `setState()` para reconstruir a interface após uma mudança de estado.
- `TextField` e `TextEditingController`.
- `Row`, `Column`, `Padding`, `SizeBox` e `EdgeInsets`.
- Conversão com `int.tryParse()` e `double.tryParse()`.

1.3 Conceitos novos neste material

- `List` e `List<String>`.
- `ListView` e `ListView.builder`.
- `itemCount`, `itemBuilder` e `index`.
- `Card` e `ListTile`.
- `leading`, `title`, `subtitle` e `trailing`.
- Criação de uma classe `Produto` e `List<Produto>`.
- `add()`, `removeAt()`, `isEmpty` e `length`.
- `Expanded` envolvendo um `ListView` dentro de uma `Column`.

2. Exercício 1 — Primeira lista com ListView

O primeiro passo é sair de uma sequência fixa de widgets em uma Column e utilizar um componente preparado para representar conteúdo vertical rolável.

Objetivos

- Compreender a diferença básica entre Column e ListView.
- Criar uma lista vertical com vários itens.
- Utilizar Divider para separar visualmente os elementos.
- Perceber que ListView permite rolagem quando o conteúdo ultrapassa a tela.

Código completo

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MeuApp());
}

class MeuApp extends StatelessWidget {
  const MeuApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: TelaDisciplinas(),
    );
  }
}

class TelaDisciplinas extends StatelessWidget {
  const TelaDisciplinas({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Minhas Disciplinas'),
      ),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: const [
          Text(
            'Programação',
            style: TextStyle(fontSize: 20),
          ),
          Divider(),
          Text(
            'Banco de Dados',
            style: TextStyle(fontSize: 20),
          ),
          Divider(),
          Text(
            'Redes de Computadores',
            style: TextStyle(fontSize: 20),
          ),
          Divider(),
          Text(
            'Flutter',
            style: TextStyle(fontSize: 20),
          ),
          Divider(),
          Text(
            'Segurança da Informação',
            style: TextStyle(fontSize: 20),
          ),
        ],
      ),
    );
  }
}
```

```

    },
  );
};
}
}

```

O que observar

A estrutura **ListView(children: [...])** lembra uma Column, mas foi criada para exibir uma sequência rolável de elementos.

COMPARAÇÃO

Column organiza elementos verticalmente. ListView também os organiza verticalmente, mas já possui comportamento de rolagem para conteúdos maiores que a área disponível.

O widget **Divider()** cria uma linha divisória entre dois itens e ajuda na leitura visual da lista.

3. Exercício 2 — List<String> e ListView.builder

No exercício anterior, cada item foi escrito manualmente. Isso funciona para poucas informações, mas não é adequado quando a quantidade de elementos vem de dados. Agora os nomes dos cursos serão armazenados em uma List<String>.

Objetivos

- Criar e percorrer uma List<String>.
- Construir a interface com ListView.builder.
- Compreender itemCount, itemBuilder, index e length.
- Relacionar uma posição da lista ao widget que será mostrado.

Código completo

```

import 'package:flutter/material.dart';

void main() {
  runApp(const MeuApp());
}

class MeuApp extends StatelessWidget {
  const MeuApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: TelaCursos(),
    );
  }
}

class TelaCursos extends StatelessWidget {
  const TelaCursos({super.key});

  @override
  Widget build(BuildContext context) {
    final List<String> cursos = [
      'Flutter',
      'Python',
      'Java',
    ];

```

```

    'Banco de Dados',
    'Redes',
    'Segurança da Informação',
    'Inteligência Artificial',
    'Desenvolvimento Web',
  ];

  return Scaffold(
    appBar: AppBar(
      title: const Text('Cursos'),
    ),
    body: ListView.builder(
      itemCount: cursos.length,
      itemBuilder: (context, index) {
        return ListTile(
          leading: const Icon(Icons.school),
          title: Text(cursos[index]),
        );
      },
    ),
  );
}

```

A lista de dados

A declaração **List<String>** indica uma lista cujos elementos são textos.

```

final List<String> cursos = [
  'Flutter',
  'Python',
  'Java',
  'Banco de Dados',
];

```

itemCount

```
itemCount: cursos.length,
```

O Flutter precisa saber quantos itens podem ser solicitados. Se a lista possuir oito elementos, `cursos.length` será igual a 8.

itemBuilder e index

```

itemBuilder: (context, index) {
  return ListTile(
    title: Text(cursos[index]),
  );
},

```

O `index` representa a posição do elemento que está sendo construído. Assim, `cursos[0]` corresponde ao primeiro curso, `cursos[1]` ao segundo e assim sucessivamente.

MODELO MENTAL

Lista de dados → index → itemBuilder → widget correspondente.

4. Exercício 3 — Cards com Card e ListTile

Com os dados já vindo de uma lista, podemos melhorar a apresentação. Cada curso será mostrado dentro de um Card e organizado por um ListTile.

Objetivos

- Reutilizar `List<String>` e `ListView.builder`.
- Introduzir `Card` e `ListTile`.
- Compreender `leading`, `title`, `subtitle` e `trailing`.
- Separar melhor os dados por meio de uma estrutura visual padronizada.

Código completo

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MeuApp());
}

class MeuApp extends StatelessWidget {
  const MeuApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: TelaCursos(),
    );
  }
}

class TelaCursos extends StatelessWidget {
  const TelaCursos({super.key});

  @override
  Widget build(BuildContext context) {
    final List<String> cursos = [
      'Flutter',
      'Python',
      'Java',
      'Banco de Dados',
      'Redes de Computadores',
      'Segurança da Informação',
    ];

    return Scaffold(
      appBar: AppBar(
        title: const Text('Cursos Disponíveis'),
      ),
      body: ListView.builder(
        padding: const EdgeInsets.all(12),
        itemCount: cursos.length,
        itemBuilder: (context, index) {
          return Card(
            margin: const EdgeInsets.only(bottom: 10),
            child: ListTile(
              leading: const Icon(
                Icons.computer,
                size: 35,
              ),
              title: Text(
                cursos[index],
                style: const TextStyle(
                  fontWeight: FontWeight.bold,
                ),
              ),
              subtitle: Text('Curso ${index + 1}'),
            ),
          );
        },
      ),
    );
  }
}
```

```

        trailing: const Icon(
          Icons.arrow_forward_ios,
        ),
      ),
    );
  },
),
);
}
}
}

```

Estrutura de um ListTile

```

ListTile(
  leading: Icon(...),
  title: Text(...),
  subtitle: Text(...),
  trailing: Icon(...),
)

```

LEITURA VISUAL

leading fica no início do item; title é a informação principal; subtitle apresenta informação complementar; trailing ocupa a extremidade final.

Neste ponto, o Card ainda é apenas uma apresentação visual. No material seguinte, o mesmo ListTile ganhará onTap e passará a executar ações ao ser tocado.

5. Exercício 4 — Lista de objetos com List<Produto>

Uma lista de String permite armazenar apenas um texto por posição. Em uma aplicação real, um item normalmente possui vários atributos. Um produto, por exemplo, pode ter nome, preço e quantidade em estoque.

Objetivos

- Criar uma classe Produto.
- Construir uma List<Produto>.
- Acessar um objeto pela posição index.
- Exibir vários atributos do mesmo objeto dentro de um Card.

Código completo

```

import 'package:flutter/material.dart';

void main() {
  runApp(const MeuApp());
}

class Produto {
  final String nome;
  final double preco;
  final int quantidade;

  Produto({
    required this.nome,
    required this.preco,
    required this.quantidade,
  });
}

```

```

}

class MeuApp extends StatelessWidget {
  const MeuApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: TelaProdutos(),
    );
  }
}

class TelaProdutos extends StatelessWidget {
  const TelaProdutos({super.key});

  @override
  Widget build(BuildContext context) {
    final List<Produto> produtos = [
      Produto(
        nome: 'Teclado',
        preco: 120.00,
        quantidade: 5,
      ),
      Produto(
        nome: 'Mouse',
        preco: 80.00,
        quantidade: 10,
      ),
      Produto(
        nome: 'Monitor',
        preco: 950.00,
        quantidade: 3,
      ),
      Produto(
        nome: 'Headset',
        preco: 250.00,
        quantidade: 7,
      ),
    ];

    return Scaffold(
      appBar: AppBar(
        title: const Text('Produtos'),
      ),
      body: ListView.builder(
        padding: const EdgeInsets.all(12),
        itemCount: produtos.length,
        itemBuilder: (context, index) {
          final Produto produto = produtos[index];

          return Card(
            child: ListTile(
              leading: const Icon(
                Icons.inventory_2,
                size: 35,
              ),
              title: Text(
                produto.nome,
                style: const TextStyle(
                  fontWeight: FontWeight.bold,
                ),
              ),
              subtitle: Text(
                'R\$ ${produto.preco.toStringAsFixed(2)} '
                '• Estoque: ${produto.quantidade}',
              ),
              trailing: const Icon(
                Icons.arrow_forward_ios,
              ),
            ),
          );
        },
      ),
    );
  }
}

```

```

    },
  ),
);
}
}
}

```

A mudança conceitual

```

final List<Produto> produtos = [
  Produto(nome: 'Teclado', preco: 120.00, quantidade: 5),
  Produto(nome: 'Mouse', preco: 80.00, quantidade: 10),
];

```

Agora cada posição da lista contém um objeto completo. O itemBuilder usa o index para recuperar o Produto correspondente:

```

final Produto produto = produtos[index];

```

Depois disso, os atributos podem ser acessados normalmente por produto.nome, produto.preco e produto.quantidade.

CONEXÃO COM ORIENTAÇÃO A OBJETOS

A lista deixa de ser apenas uma coleção de textos e passa a representar entidades do domínio da aplicação. Esse mesmo objeto Produto será usado no material seguinte para passagem de dados entre telas.

6. Exercício 5 — Cadastro dinâmico de produtos

Agora reunimos os conteúdos anteriores com os conceitos de estado e entrada de dados. A lista começará vazia e poderá ser modificada pelo usuário durante a execução.

Objetivos

- Manter uma List<Produto> como estado da tela.
- Adicionar novos objetos com add().
- Remover objetos com removeAt().
- Atualizar a lista visual usando setState().
- Mostrar um estado vazio com isEmpty.
- Usar Expanded para permitir que ListView ocupe o espaço restante de uma Column.

Código completo

```

import 'package:flutter/material.dart';

void main() {
  runApp(const MeuApp());
}

class Produto {
  final String nome;
  final double preco;

  Produto({

```

```

        required this.nome,
        required this.preco,
    });
}

class MeuApp extends StatelessWidget {
  const MeuApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: TelaProdutos(),
    );
  }
}

class TelaProdutos extends StatefulWidget {
  const TelaProdutos({super.key});

  @override
  State<TelaProdutos> createState() =>
    _TelaProdutosState();
}

class _TelaProdutosState extends State<TelaProdutos> {
  final TextEditingController nomeController =
    TextEditingController();

  final TextEditingController precoController =
    TextEditingController();

  final List<Produto> produtos = [];

  void adicionarProduto() {
    final String nome = nomeController.text.trim();
    final double preco =
      double.tryParse(precoController.text) ?? 0;

    if (nome.isEmpty || preco <= 0) {
      return;
    }

    setState(() {
      produtos.add(
        Produto(
          nome: nome,
          preco: preco,
        ),
      );
    });

    nomeController.clear();
    precoController.clear();
  }

  void removerProduto(int index) {
    setState(() {
      produtos.removeAt(index);
    });
  }

  void limparCampos() {
    nomeController.clear();
    precoController.clear();
  }

  @override
  void dispose() {
    nomeController.dispose();
    precoController.dispose();
    super.dispose();
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Cadastro de Produtos'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        children: [
          TextField(
            controller: nomeController,
            decoration: const InputDecoration(
              labelText: 'Nome do produto',
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 12),
          TextField(
            controller: precoController,
            keyboardType: TextInputType.number,
            decoration: const InputDecoration(
              labelText: 'Preço',
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 15),
          Row(
            mainAxisAlignment:
              MainAxisAlignment.center,
            children: [
              ElevatedButton(
                onPressed: adicionarProduto,
                child: const Text('Adicionar'),
              ),
              const SizedBox(width: 15),
              ElevatedButton(
                onPressed: limparCampos,
                child: const Text('Limpar'),
              ),
            ],
          ),
          const SizedBox(height: 20),
          Expanded(
            child: produtos.isEmpty
              ? const Center(
                  child: Text(
                    'Nenhum produto cadastrado.',
                  ),
                )
              : ListView.builder(
                  itemCount: produtos.length,
                  itemBuilder: (context, index) {
                    final Produto produto =
                      produtos[index];

                    return Card(
                      child: ListTile(
                        leading: const Icon(
                          Icons.inventory_2,
                        ),
                        title: Text(produto.nome),
                        subtitle: Text(
                          'R\$ ${produto.preco.toStringAsFixed(2)}',
                        ),
                        trailing: IconButton(
                          icon: const Icon(
                            Icons.delete,
                          ),
                          onPressed: () {
                            removerProduto(index);
                          },
                        ),
                      ),
                    );
                  },
                ),
          ),
        ],
      ),
    ),
  );
}

```

```

    },
  ),
);
},
),
),
),
),
);
}
}

```

1. A lista como estado

```
final List<Produto> produtos = [];
```

A coleção começa vazia. Diferentemente dos exercícios anteriores, agora ela será alterada durante a execução.

2. Adicionando um objeto

```

setState() {
  produtos.add(
    Produto(nome: nome, preco: preco),
  );
}

```

`add()` modifica a coleção e `setState()` informa ao Flutter que a interface precisa ser reconstruída. O novo Card aparece porque `ListView.builder` passa a enxergar um elemento a mais em `produtos.length`.

3. Removendo um objeto

```

setState() {
  produtos.removeAt(index);
}

```

`removeAt(index)` remove o elemento da posição selecionada. Após o `setState()`, a lista é reconstruída sem aquele item.

4. Estado vazio

```

produtos.isEmpty
? const Center(
  child: Text('Nenhum produto cadastrado.'),
)
: ListView.builder(...)

```

Uma interface precisa prever o que será mostrado quando ainda não há dados. `isEmpty` permite tratar esse estado explicitamente.

5. Por que usar Expanded?

```

Column(
  children: [
    // campos e botões

```

```
Expanded(
  child: ListView.builder(...),
),
],
)
```

ATENÇÃO AO LAYOUT

Uma ListView vertical dentro de uma Column precisa receber uma altura limitada. Expanded faz a lista ocupar o espaço restante da tela e evita o conflito de altura ilimitada.

Fluxo completo do exercício

FLUXO

TextField → controller → Produto → produtos.add() → setState() → ListView.builder → Card. Para excluir: IconButton → removeAt(index) → setState() → Card desaparece.

7. Síntese dos conceitos

Elemento	Função principal
List<T>	Coleção de elementos do mesmo tipo.
length	Quantidade de elementos da lista.
isEmpty	Indica se a coleção está vazia.
ListView	Lista rolável com filhos fornecidos diretamente.
ListView.builder	Constrói itens a partir de uma função builder.
itemCount	Informa quantos itens podem ser construídos.
itemBuilder	Transforma cada posição da coleção em um widget.
index	Posição atual solicitada pelo builder.
Card	Agrupa visualmente informações relacionadas.
ListTile	Estrutura pronta para leading, title, subtitle e trailing.
List<Produto>	Coleção de objetos do domínio da aplicação.
add()	Adiciona um elemento ao final da lista.
removeAt()	Remove o elemento da posição indicada.

Elemento	Função principal
Expanded	Entrega ao filho o espaço restante dentro de Row/Column.
setState()	Reconstrói a interface após alteração do estado.

8. O que o aluno deve perceber

Ao final deste bloco, a interface deixa de ser uma sequência de widgets isolados e passa a representar diretamente uma coleção de dados. Isso prepara o aluno para um passo decisivo: tornar cada Card interativo e usar o item selecionado para abrir outra tela.

PONTE PARA O PRÓXIMO MATERIAL

O próximo passo começa exatamente aqui: adicionar onTap ao ListTile, mostrar feedback com SnackBar e usar Navigator.push()/pop() para transformar a lista em um fluxo de aplicação.

9. Exercícios de fixação

Os exercícios abaixo podem ser utilizados ao final da aula ou como prática complementar. Eles não introduzem navegação; trabalham apenas os conceitos deste material.

1. Lista de linguagens

Crie uma List<String> com pelo menos oito linguagens e apresente-as com ListView.builder e ListTile.

2. Catálogo de livros

Crie uma classe Livro com título e autor. Apresente uma List<Livro> em Cards.

3. Lista de alunos

Crie uma classe Aluno com nome e nota. Mostre nome no title e nota no subtitle.

4. Cadastro de tarefas

Permita cadastrar uma tarefa em uma lista dinâmica e removê-la por meio de um IconButton.

5. Estoque simples

Crie Produto com nome, preço e quantidade. Cadastre itens e exiba os dados em Cards; a lista deve mostrar uma mensagem quando estiver vazia.

LIMITE DESTA ETAPA

Ainda não utilizamos onTap, Navigator, múltiplas telas, Form ou TextFormField. Esses recursos pertencem ao próximo material.